
Consumo de APIs em JavaScript

Um guia completo sobre fundamentos, protocolos e implementação prática para o desenvolvimento de sistemas modernos.

Fundamentos de APIs

O que é uma API?

Interface de Programação de Aplicações que permite a comunicação entre diferentes sistemas de forma padronizada e segura.

Arquitetura REST

Modelo que utiliza os protocolos da Web (HTTP) para transferência de dados, baseando-se em recursos e estados.



REQUISIÇÃO (JSON)



RESPOSTA (JSON)

HTTP e Endpoints

PRINCIPAIS MÉTODOS HTTP

GET

Consulta dados

POST

Cria novos dados

PUT

Atualiza dados

DELETE

Remove dados

ANATOMIA DE UM ENDPOINT

O endereço único onde o recurso pode ser acessado:

```
https://api.exemplo.com/v1/usuarios/123
```

Base URL

Versão

Recurso

Identificador

Formato JSON

O que é JSON?

JavaScript Object Notation: O padrão universal para troca de dados.

- Leve e fácil de ler por humanos
- Independente de linguagem
- Estrutura baseada em Chave e Valor
- Suporta Objetos, Arrays e Tipos Primitivos

```
{  
  "id": 1,  
  "nome": "João Silva",  
  "ativo": true,  
  "habilidades": [  
    "JavaScript",  
    "APIs"  
  ],  
  "endereco": {  
    "cidade": "São Paulo"  
  }  
}
```

Consumo em JavaScript

1. fetch()

```
fetch('https://api.exemplo.com/dados')
```

2. Promises

```
.then(response => response.json())
```

3. Async / Await

```
async function getData() {  
  const res = await fetch(url);  
  const data = await res.json();  
}
```

Dados e Tratamento de Erros

Manipulação de Dados

```
// Conversão e Filtro
const lista = data.map(item => ({
  nome: item.name,
  id: item.id
}));

const ativos = lista.filter(
  i => i.status === 'active'
);
```

Tratamento de Erros

```
try {
  const res = await fetch(url);
  if (!res.ok)
    throw new Error('Falha');
} catch (error) {
  console.error("Erro no consumo:",
    error.message);
}
```

Aplicação com API Pública

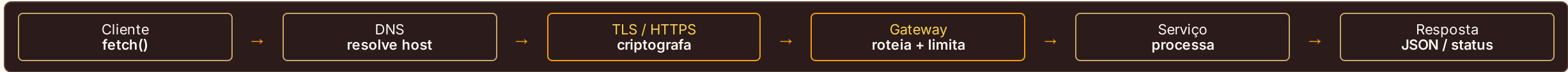


Projeto Prático Final

Consolidação do conhecimento através da criação de um buscador de dados em tempo real, integrando interface de usuário, consumo assíncrono e tratamento dinâmico de informações.

Comunicação de APIs: do DNS ao dado

Uma chamada de API atravessa várias camadas: rede, segurança, roteamento, contrato e processamento de negócio.



CAMADA DE TRANSPORTE

O que acontece antes do código?

- ◆ **DNS:** converte `api.exemplo.com` em um endereço IP.
- ◆ **TCP:** estabelece uma conexão confiável; HTTP/2 pode reutilizá-la.
- ◆ **TLS:** valida o certificado e negocia criptografia no HTTPS.
- ◆ **Proxy/CDN:** pode cachear, filtrar e encaminhar o tráfego.

CONTRATO E EXECUÇÃO

Gateway até o serviço

- ◆ Gateway valida token, origem, rota e **rate limit**.
- ◆ Roteamento envia a chamada ao serviço responsável.
- ◆ O serviço valida schema, executa regras e acessa dados.
- ◆ Headers carregam rastreo, cache e negociação de formato.

```
traceparent: 00-4bf92f...
Cache-Control: max-age=60
Content-Type: application/json
```

CONFIABILIDADE

Falhas não são exceção

- ◆ **Timeout:** encerra esperas indefinidas.
- ◆ **Retry:** repete apenas falhas transitórias.
- ◆ **Backoff:** aumenta o intervalo entre tentativas.
- ◆ **Idempotência:** evita duplicar uma operação POST.
- ◆ **Circuit breaker:** interrompe chamadas a serviço degradado.

NEGOCIAÇÃO DE DADOS

Como o formato é combinado?

Accept

Cliente informa o formato esperado.

Content-Type

Indica o formato enviado.

Serialize

Objeto vira bytes/JSON.

Deserialize

Resposta volta a objeto.

ALÉM DO REQUEST/RESPONSE

Comunicação assíncrona

- ◆ **Polling:** cliente consulta periodicamente.
- ◆ **Webhook:** servidor notifica uma URL do cliente.
- ◆ **Fila/evento:** produtores e consumidores ficam desacoplados.
- ◆ **WebSocket/SSE:** conexão persistente para atualização em tempo real.

EVOLUÇÃO DA API

Compatibilidade e observabilidade

- ◆ Versionar contratos sem quebrar clientes existentes.
- ◆ Correlacionar logs por **request ID** ou trace ID.
- ◆ Medir latência, taxa de erro, throughput e saturação.
- ◆ Documentar endpoints com OpenAPI/Swagger.

Modelo mental: API confiável é um sistema distribuído — o sucesso depende de rede segura, contrato versionado, autenticação, processamento, observabilidade e estratégias para falhas.